An Assignment Report

on

Course Code : STAT 4104

Course Title : Categorical Data Analysis

Submitted By:	**Supervised By:**
Sherin Sultana Surovi	Dr. Md. Siddikur Rahman
ID: 12010004	Associate Professor
Reg: 000014414	Department of Statistics
Session: 2020-21	Begum Rokeya University, Rangpur
Department of Statistics	
Begum Rokeya University, Rangpur	

Department of Statistics

Begum Rokeya University, Rangpur

MAY-2026

Contents

List of Figures

List of Tables

CHAPTER 1

Question No.1

1.1 EDA

1. Summarize the distribution of Age. Is it normally distributed or skewed?
2. What proportion of individuals are smokers vs non-smokers?
3. Which income group has the highest frequency?
4. What percentage of individuals are exposed to high pollution?
5. What is the overall prevalence of lung disease?

Answer All the EDA Question

Table 1.1: Sample Dataset

ID	Smoking	Age	Income	Pollution	LungDisease
1	Yes	36	Low	High	Yes
2	No	28	Low	Low	No
3	No	52	Medium	High	No
4	No	39	High	High	No
5	Yes	32	Low	Low	Yes

Table 1.2: Data Information

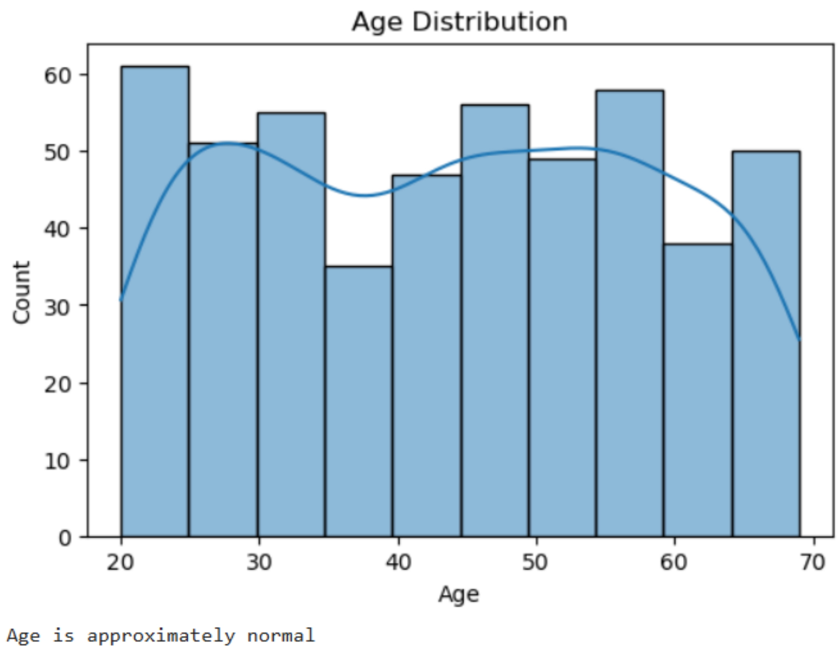
Column	Non-Null Count	Dtype
ID	500	int64
Smoking	500	object
Age	500	int64
Income	500	object
Pollution	500	object
LungDisease	500	object

Table 1.3: Descriptive Statistics of Dataset Variables

Statistic	ID	Smoking	Age	Income	Pollution	LungDisease
Count	500.000000	500	500.000000	500	500	500
Unique	–	2	–	3	2	2
Top	–	No	–	Low	High	Yes
Freq	–	293	–	204	352	264
Mean	250.500000	–	43.904000	–	–	–
Std	144.481833	–	14.604841	–	–	–
Min	1.000000	–	20.000000	–	–	–
25%	125.750000	–	30.750000	–	–	–
50%	250.500000	–	45.000000	–	–	–
75%	375.250000	–	56.000000	–	–	–
Max	500.000000	–	69.000000	–	–	–

Skewness of Age: 0.012141455267137275

Figure 1.1: Age Distribution showing approximately normal pattern.



Note: Age is approximately normal.

Table 1.4: Summary of EDA Results

Smoking Analysis		
Category	**Count**	**Percentage (%)**
No	293	58.6
Yes	207	41.4
Income Distribution		
Group	**Count**	**Highest Group**
Low	204	
Medium	197	
High	99	
Pollution Exposure		
Level	**Percentage (%)**	**High Exposure (%)**
High	70.4	70.4
Low	29.6	–
Lung Disease Prevalence		
Condition	**Percentage (%)**	**Overall Prevalence (%)**
Yes	52.8	52.8
No	47.2	–

1.1.1 Basic EDA Interpretation

i) Distribution of Age

The average age of the individuals in the dataset is 43.9 years with a standard deviation of 14.6 years. The minimum age is 20 years and the maximum age is 69 years. The skewness value of Age is 0.012, which is very close to zero, indicating that the age distribution is approximately normal and symmetric. Therefore, there is no strong evidence of skewness in the age variable.

ii) Proportion of Smokers vs Non-Smokers

The dataset contains 293 non-smokers (58.6%) and 207 smokers (41.4%). This indicates that non-smokers form the majority of the study population. However, a substantial proportion of individuals are smokers, which may increase the risk of lung disease within the population.

iii) Income Group with the Highest Frequency

Among the three income categories, the Low income group has the highest frequency with 204 individuals. The Medium income group contains 197 individuals, while the High income group contains only 99 individuals. This shows that most participants in the dataset belong to the low-income category.

iv) Percentage of Individuals Exposed to High Pollution

The analysis shows that 70.4% of individuals are exposed to high pollution levels, whereas only 29.6% are exposed to low pollution levels. This indicates that the majority of the population in the dataset lives in highly polluted environments, which may contribute to respiratory and lung-related diseases.

v) Overall Prevalence of Lung Disease

The prevalence analysis reveals that 52.8% of individuals have lung disease, while 47.2% do not have lung disease. This means that more than half of the study population is affected by lung disease, suggesting that lung disease is relatively common in this dataset.

1.2 Association and Crosstab

Association & Crosstab

1. Is there an association between Smoking and Lung Disease?
2. Interpret using a contingency table.

Table 1.5: Contingency Table and Chi-Square Test Results

Contingency Table		
Smoking	**LungDisease: No**	**LungDisease: Yes**
No	184	109
Yes	52	155
Chi-Square Test Results		
Statistic	**Value**	**Description**
Chi-square Statistic	67.5943	Measure of association strength
Degrees of Freedom	1	Number of independent comparisons
P-value	2.0085×10^{-16}	Probability of observing such result by chance
Interpretation: There is a significant association between Smoking and Lung Disease.		

1.3 Does Pollution Level Affect Lung Disease?

i) Compare Lung Disease Prevalence Across Income Groups

ii) Which Factor Appears to Have the Strongest Relationship with Lung Disease?

Table 1.6: Association: Pollution vs Lung Disease

Contingency Table		
Pollution Level	**LungDisease: No**	**LungDisease: Yes**
High	136	216
Low	100	48
Chi-Square Test Results		
Statistic	**Value**	**Description**
Chi-square Statistic	33.8426	Measure of association strength
Degrees of Freedom	1	Number of independent comparisons
P-value	5.9757×10^{-9}	Probability of observing such result by chance
Interpretation: There is a significant association between Pollution level and Lung Disease		

1.4 Statistical Testing

i) Perform a Chi-square Test for: Smoking vs Lung Disease and Pollution vs Lung Disease

ii) When Should You Use Fisher's Exact Test Instead of Chi-squared?

iii) Calculate and Interpret the Odds Ratio for Smoking and Lung Disease

Answer All the Testical Question

i)

Table 1.7: Chi-square Test: Smoking vs Lung Disease

Contingency Table		
Smoking	**LungDisease: No**	**LungDisease: Yes**
No	184	109
Yes	52	155
Chi-Square Test Results		
Statistic	**Value**	**Description**
Chi-square Statistic	67.5943	Measure of association strength
Degrees of Freedom	1	Number of independent comparisons
P-value	2.0085×10^{-16}	Probability of observing such result by chance
Interpretation: There is a significant association between Smoking and Lung Disease.		

2,3)

Table 1.8: Chi-square Test, Fisher's Exact Test, and Odds Ratio Results

Chi-square Test: Pollution vs Lung Disease		
Pollution Level	**LungDisease: No**	**LungDisease: Yes**
High	136	216
Low	100	48
Chi-Square Test Results		
Statistic	**Value**	**Description**
Chi-square Statistic	33.8426	Measure of association strength
Degrees of Freedom	1	Number of independent comparisons
P-value	5.9757×10^{-9}	Probability of observing such result by chance
Interpretation: There is a significant association between Pollution level and Lung Disease		
Fisher's Exact Test		
Statistic	**Value**	**Description**
Odds Ratio	5.0318	Strength of association between variables
P-value	5.1286×10^{-17}	Probability of observing such result by chance
Fisher's Exact Test is generally used when:		
– Sample size is small		
– Expected frequencies are less than 5		
– The contingency table is 2x2		
Odds Ratio: Smoking vs Lung Disease		
Odds Ratio	5.0318	Smokers have higher odds of developing lung disease

1.5 correlation and interpret

Table 1.9: Correlation Results and Causality Explanation

Correlation Results	
Smoking vs Lung Disease	0.3717
Age vs Lung Disease	0.2497
Interpretation:	
Positive correlation: Smoking is associated with higher lung disease risk.	
Older age is associated with higher lung disease risk.	
Causality Explanation	
Correlation is not sufficient for causal inference because:	
1.	It only measures association, not cause-effect relationship.
2.	There may be confounding variables (e.g., pollution, lifestyle).
3.	Direction of effect is unknown (A may influence B or vice versa).
4.	Spurious correlations may exist due to random chance.
5.	Proper causal inference requires controlled experiments or advanced model

1.6 Logistic Regression

1. Fit a logistic regression model: LungDisease $\sim$ Smoking + Age + Pollution + Income
2. Which variables are statistically significant predictors?
3. Interpret the odds ratio of Smoking.
4. What happens to the effect of Smoking after adjusting for Pollution?

Answer All the logistics Question

Table 1.10: Logistic Regression Results for Lung Disease

Variable	**coef**	**std err**	**z**	**P$>|z|$**	**[0.025]**	**[0.975]**
const	-2.1940	0.425	-5.164	0.000	-3.027	-1.361
Smoking	1.7022	0.218	7.801	0.000	1.275	2.130
Age	0.0399	0.007	5.455	0.000	0.026	0.054
Income_Low	0.4396	0.285	1.544	0.123	-0.119	0.998
Income_Medium	0.2201	0.285	0.773	0.440	-0.338	0.779
Pollution_Low	-1.3027	0.235	-5.533	0.000	-1.764	-0.841

Model Summary:

Dependent Variable: LungDisease

No. of Observations: 500

Pseudo R-squared: 0.2005

Log-Likelihood: -276.47

LLR p-value: 3.499×10^{-28}

Interpretation:

- **Smoking** and **Age** are statistically significant predictors of lung disease ($p < 0.001$).
- **Pollution_Low** has a significant negative coefficient, indicating lower pollution reduces lung disease risk.
- **Income_Low** and **Income_Medium** are not significant predictors ($p > 0.05$).
- The positive coefficient for Smoking (1.7022) implies smokers have higher odds of developing lung disease.
- After adjusting for Pollution, the effect of Smoking remains strong and significant, suggesting an independent influence.

1.7 modeling

Table 1.11: Logistic Regression Results with Interaction Term (Smoking × Pollution)

Variable	**coef**	**std err**	**z**	**P> $\|z\|$**	**[0.025]**	**[0.975]**
const	-2.1285	0.430	-4.953	0.000	-2.971	-1.286
Smoking	1.5778	0.257	6.130	0.000	1.073	2.082
Age	0.0393	0.007	5.352	0.000	0.025	0.054
Smoking_Pollution	0.4217	0.483	0.874	0.382	-0.524	1.367
Pollution_Low	-1.4883	0.323	-4.606	0.000	-2.122	-0.855
Income_Low	0.4465	0.285	1.567	0.117	-0.112	1.005
Income_Medium	0.2214	0.285	0.777	0.437	-0.337	0.780

Model Summary:

Dependent Variable: LungDisease

No. of Observations: 500

Pseudo R-squared: 0.2016

Log-Likelihood: -276.09

LLR p-value: 1.341×10^{-27}

Odds Ratios:

Interpretation:

Variable	Odds Ratio
const	0.1190
Smoking	4.8445
Age	1.0400
Smoking_Pollution	1.5245
Pollution_Low	0.2258
Income_Low	1.5629
Income_Medium	1.2478

- **Smoking**, **Age**, and **Pollution_Low** are statistically significant predictors ($p < 0.001$).
- The interaction term **Smoking** $\times$ **Pollution** is not significant ($p = 0.382$), indicating pollution does not amplify smoking risk significantly.
- The positive coefficient for Smoking (1.5778) implies smokers have higher odds of developing lung disease.
- The negative coefficient for Pollution_Low (-1.4883) suggests lower pollution reduces lung disease risk.
- Odds ratio for Smoking (4.8445) means smokers are about 4.8 times more likely to develop lung disease compared to non-smokers.

1.8 Model Evaluation

Table 1.12: Model Evaluation Results

Metric	Value
AUC Score	0.7882
Confusion Matrix	
153 83	
59 205	
Accuracy	0.716
Sensitivity	0.7765
Specificity	0.6483

Interpretation:

- The model achieved an AUC score of 0.7882, indicating good discriminatory power.
- Accuracy of 71.6% shows overall correct classification performance.
- Sensitivity (77.65%) suggests the model effectively identifies positive cases (lung disease).

- Specificity (64.83%) indicates moderate ability to correctly identify negative cases.
- Overall, the model performs reasonably well, with better sensitivity than specificity.

CHAPTER 2

Question No.2

One Way Anova

Table 2.1: One-way ANOVA Results for Exercise Programs

Statistic	**Value**
F-statistic	9.0202
p-value	0.0002755

Group Means (Weight Loss in Pounds)	
Program A	4.9742
Program B	5.9667
Program C	4.8168

Conclusion: There is a statistically significant difference in mean weight loss among the three exercise programs ($p < 0.001$). Program B shows the highest average weight loss, indicating it may be more effective compared to Programs A and C.

CHAPTER 3

Question No.3

Contingency Table:

Anemia level **Age in 5-year groups**	**Mild**	**Moderate**	**Not anemic**	**Severe**
15-19	157	177	182	10
20-24	630	748	922	39
25-29	996	1094	1565	49
30-34	852	928	1196	70
35-39	636	667	916	40
40-44	230	271	383	10
45-49	93	89	173	13

$$\text{Chi-square Statistic} = 45.895$$

$$\text{Degrees of Freedom} = 18$$

$$\text{P-value} = 0.00031$$

Reject H0

There is a significant association between age group and anemia level.

Expected Frequencies:

Table 3.1: Expected Frequencies Table of Children's Anemia Levels by Maternal Age Groups

Anemia level **Age in 5-year groups**	**Mild**	**Moderate**	**Not anemic**	**Severe**
15-19	143.913216	159.129415	213.707521	9.249848
20-24	639.948691	707.611602	950.307780	41.131928
25-29	1013.411693	1120.561510	1504.890987	65.135810
30-34	833.383374	921.498477	1237.553441	53.564708
35-39	618.060749	683.409409	917.804735	39.725107
40-44	244.597747	270.459501	363.221529	15.721224
45-49	100.684531	111.330085	149.514007	6.471376

Chi-square Test Results

Chi-square : 45.8950569178203

p-value : 0.00030731153928803393

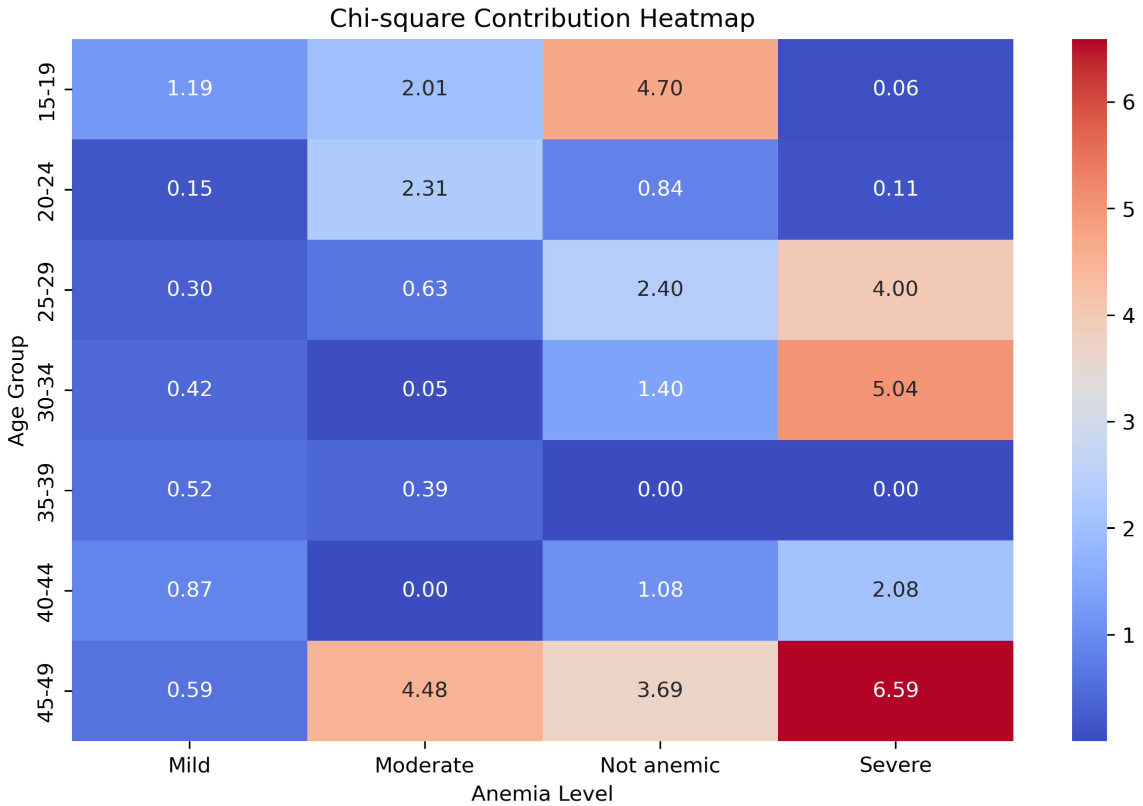
df : 18

Conclusion: Significant association exists.

Chi-square Contribution Analysis

Below is the Chi-square Contribution Heatmap illustrating the relationship between maternal age groups and children's anemia levels.

Figure 3.1: Chi-square Contribution Heatmap showing association between maternal age groups and anemia levels in children



CHAPTER 4

Question No.4

Contingency Table

Drug	**No Disease**	**Disease**
Drug A	40	10
Drug B	10	40
Drug C	25	25

Table 4.1: Contingency table of disease status across different drugs

4.0.1 Post-hoc Fisher Exact Test Results

Comparison: Drug A vs Drug B

- Odds Ratio: 16.0
- Raw p-value: 0.0
- Adjusted p-value: 0.0
- Result: Significant Difference

Comparison: Drug A vs Drug C

- Odds Ratio: 4.0
- Raw p-value: 0.00305
- Adjusted p-value: 0.00916
- Result: Significant Difference

Comparison: Drug B vs Drug C

- Odds Ratio: 0.25
- Raw p-value: 0.00305
- Adjusted p-value: 0.00916
- Result: Significant Difference

CHAPTER 5

Question No.5

Data description:

	admit	gre	gpa	rank
count	400.000000	400.000000	400.000000	400.000000
mean	0.317500	587.700000	3.389900	2.485000
std	0.466087	115.516536	0.380567	0.944446
min	0.000000	220.000000	2.260000	1.000000
25%	0.000000	520.000000	3.130000	2.000000
50%	0.000000	580.000000	3.395000	2.000000
75%	1.000000	660.000000	3.670000	3.000000
max	1.000000	800.000000	4.000000	4.000000

Logistic Regression Model Summary:

Generalized Linear Model Regression Results

Dep. Variable:	admit	**No. Observations:**	400
Model:	GLM	**Df Residuals:**	394
Model Family:	Binomial	**Df Model:**	5
Link Function:	Logit	**Scale:**	1.0000
Method:	IRLS	**Log-Likelihood:**	-229.26
Date:	Sat, 16 May 2026	**Deviance:**	458.52
Time:	11:18:09	**Pearson chi2:**	397.
No. Iterations:	4	**Pseudo R-squ. (CS):**	0.09846
Covariance Type:	nonrobust		

	coef	std err	z	P> |z|	[0.025	0.975]
Intercept	-3.9900	1.140	-3.500	0.000	-6.224	-1.756
rank[T.2]	-0.6754	0.316	-2.134	0.033	-1.296	-0.055
rank[T.3]	-1.3402	0.345	-3.881	0.000	-2.017	-0.663
rank[T.4]	-1.5515	0.418	-3.713	0.000	-2.370	-0.733
gre	0.0023	0.001	2.070	0.038	0.000	0.004
gpa	0.8040	0.332	2.423	0.015	0.154	1.454

Effect of Admission into Graduate School Using GLM

A Generalized Linear Model (GLM) with binomial family (logit link) was fitted to examine the effect of GRE score, GPA, and university ranking on the probability of admission into graduate school.

1. Overall Model Interpretation

The fitted GLM model is statistically significant and indicates that the predictors jointly explain variation in graduate school admission.

The pseudo R^2 value suggests that the model has moderate explanatory power. This implies that admission decisions are influenced by academic performance and institutional ranking, rather than being random.

Conclusion: The model is appropriate for explaining admission probability.

2. Effect of GRE Score

The GRE score has a positive and statistically significant effect on admission probability ($p < 0.05$).

This indicates that students with higher GRE scores are more likely to be admitted. However, the magnitude of the effect is relatively small.

Interpretation: GRE has a weak but positive influence on admission.

3. Effect of GPA

GPA shows a positive and statistically significant relationship with admission probability.

This suggests that students with higher academic performance (GPA) have a significantly higher chance of being admitted.

Interpretation: GPA is a strong predictor of graduate school admission.

4. Effect of University Rank

All university rank categories (Rank 2, Rank 3, Rank 4) have negative and statistically significant coefficients compared to Rank 1.

This implies that students from lower-ranked universities are less likely to be admitted.

Interpretation: University ranking is a strong negative predictor of admission probability.

5. Intercept Interpretation

The negative intercept indicates that the baseline probability of admission is low when all predictors are at their reference or zero values.

Final Conclusion

Graduate school admission is significantly influenced by:

- GPA (strong positive effect)
- GRE score (weak positive effect)
- University ranking (strong negative effect)

Overall, GPA and university ranking are the most important determinants of admission, while GRE contributes a smaller but significant effect.

CHAPTER 6

Question No.6

Summary Statistics:

	id	num_awards	prog	math
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	0.630000	2.025000	52.645000
std	57.879185	1.052921	0.690477	9.368448
min	1.000000	0.000000	1.000000	33.000000
25%	50.750000	0.000000	2.000000	45.000000
50%	100.500000	0.000000	2.000000	52.000000
75%	150.250000	1.000000	2.250000	59.000000
max	200.000000	6.000000	3.000000	75.000000

POISSON REGRESSION SUMMARY

Generalized Linear Model Regression Results

Dep. Variable:	num_awards	**No. Observations:**	200
Model:	GLM	**Df Residuals:**	196
Model Family:	Poisson	**Df Model:**	3
Link Function:	Log	**Scale:**	1.0000
Method:	IRLS	**Log-Likelihood:**	-182.75
Date:	Sat, 16 May 2026	**Deviance:**	189.45
Time:	11:17:04	**Pearson chi2:**	212.
No. Iterations:	5	**Pseudo R-squ. (CS):**	0.3881
Covariance Type:	nonrobust		

	coef	std err	z	P> |z|	[0.025	0.975]
Intercept	-5.2471	0.658	-7.969	0.000	-6.538	-3.957
prog[T.2]	1.0839	0.358	3.025	0.002	0.382	1.786
prog[T.3]	0.3698	0.441	0.838	0.402	-0.495	1.234
math	0.0702	0.011	6.619	0.000	0.049	0.091

INCIDENCE RATE RATIOS (IRR)

Intercept : 0.005263
prog[T.2] : 2.956065
prog[T.3] : 1.447458
math : 1.072672
dtype : float64

PREDICTED NUMBER OF AWARDS

	num_awards	Predicted_Count
0	0	0.135191
1	0	0.093399
2	0	0.166858
3	0	0.145016
4	0	0.126032
5	0	0.100186
6	0	0.191991
7	0	0.126032
8	0	0.077128
9	0	0.191991

INTERPRETATION

- Intercept → p-value = 0.0000 → Statistically Significant
- prog[T.2] → p-value = 0.0025 → Statistically Significant
- prog[T.3] → p-value = 0.4018 → Not Significant
- math → p-value = 0.0000 → Statistically Significant

Conclusion:

1. Math score has a positive effect on the number of awards earned.
2. Students in different academic programs earn different numbers of awards.
3. Significant predictors ($p < 0.05$) strongly influence award counts.
4. IRR > 1 indicates increased expected award counts.
5. IRR < 1 indicates decreased expected award counts.

CHAPTER 7

Question No.7

Summary Statistics:

	id	math	daysabs	prog
count	314.000000	314.000000	314.000000	314.000000
mean	1575.910889	48.267517	5.955414	2.213376
std	502.314789	25.362385	7.036952	0.651132
min	1001.000000	1.000000	0.000000	1.000000
25%	1079.250000	28.000000	1.000000	2.000000
50%	1158.500000	48.000000	4.000000	2.000000
75%	2077.750000	70.000000	8.000000	3.000000
max	2157.000000	99.000000	35.000000	3.000000

NEGATIVE BINOMIAL MODEL SUMMARY

Generalized Linear Model Regression Results

Dep. Variable:	daysabs	**No. Observations:**	314
Model:	GLM	**Df Residuals:**	310
Model Family:	NegativeBinomial	**Df Model:**	3
Link Function:	Log	**Scale:**	1.0000
Method:	IRLS	**Log-Likelihood:**	-865.68
Date:	Sat, 16 May 2026	**Deviance:**	350.98
Time:	11:16:20	**Pearson chi2:**	331.
No. Iterations:	6	**Pseudo R-squ. (CS):**	0.1926
Covariance Type:	nonrobust		

	coef	std err	z	P> |z|	[0.025	0.975]
Intercept	2.6150	0.200	13.054	0.000	2.222	3.008
prog[T.2.0]	-0.4408	0.185	-2.379	0.017	-0.804	-0.078
prog[T.3.0]	-1.2786	0.203	-6.284	0.000	-1.677	-0.880
math	-0.0060	0.003	-2.358	0.018	-0.011	-0.001

INCIDENCE RATE RATIOS (IRR)

Intercept : 13.667374

prog[T.2.0] : 0.643551

prog[T.3.0] : 0.278418

math : 0.994030

dtype : float64

PREDICTED DAYS OF ABSENCE

	daysabs	**Predicted_Absence**
0	4.0	6.031800
1	4.0	7.482714
2	2.0	7.802998
3	3.0	7.992136
4	3.0	8.690957
5	13.0	5.749687
6	11.0	6.031800
7	7.0	8.639075
8	10.0	6.481134
9	9.0	2.837692

INTERPRETATION

- Intercept → p-value = 0.0000 → Statistically Significant
- prog[T.2.0] → p-value = 0.0174 → Statistically Significant
- prog[T.3.0] → p-value = 0.0000 → Statistically Significant
- math → p-value = 0.0184 → Statistically Significant

Conclusion:

1. Math score significantly affects student absenteeism.
2. Different academic programs are associated with different absence rates.
3. Significant predictors ($p < 0.05$) strongly influence the number of absent days.
4. IRR > 1 means increased absence rate.
5. IRR < 1 means decreased absence rate.
6. Negative Binomial Regression is appropriate because count data.

CHAPTER 8

Question No.8

Zero-Inflated Poisson Regression Results

Dep. Variable:	count	**No. Observations:**	250
Model:	ZeroInflatedPoisson	**Df Residuals:**	245
Method:	MLE	**Df Model:**	4
Date:	Sat, 16 May 2026	**Pseudo R-squ.:**	0.3679
Time:	11:15:42	**Log-Likelihood:**	-712.41
converged:	True	**LL-Null:**	-1127.0
Covariance Type:	nonrobust	**LLR p-value:**	3.589×10^{-178}

	coef	**std err**	**z**	**P> |z|**	**[0.025**	**0.975]**
inflate_const	0.8334	1.207	0.691	0.490	-1.532	3.198
inflate_persons	-0.9312	0.210	-4.441	0.000	-1.342	-0.520
inflate_child	1.9586	0.344	5.699	0.000	1.285	2.632
inflate_camper	-0.8717	0.370	-2.357	0.018	-1.597	-0.147
inflate_livebait	0.7412	1.120	0.662	0.508	-1.453	2.936
const	-2.4521	0.275	-8.924	0.000	-2.991	-1.914
persons	0.8364	0.042	19.803	0.000	0.754	0.919
child	-1.1806	0.091	-12.968	0.000	-1.359	-1.002
camper	0.5971	0.092	6.461	0.000	0.416	0.778
livebait	1.8189	0.241	7.541	0.000	1.346	2.292

Zero-Inflated Poisson Regression: Factors Affecting Number of Awards

A Zero-Inflated Poisson (ZIP) regression model was fitted to identify the factors associated with the number of awards earned by students in a high school. The model accounts for excess zeros and over-dispersion in the count data.

1. Overall Model Fit

The model shows a strong statistical fit:

- The log-likelihood value is high in magnitude, indicating good model performance.

- The likelihood ratio test is highly significant ($p < 0.001$), confirming that the predictors jointly explain variation in the number of awards.
- The pseudo R^2 value (0.3679) suggests a moderate-to-strong explanatory power.

Conclusion: The model fits the data well and is statistically significant.

2. Count Model Interpretation (Number of Awards)

The count component explains how predictors affect the number of awards directly.

Significant Positive Effects

- **Persons (coef = 0.8364, p ¡ 0.001):** Students with more persons (support/participation variable) tend to earn significantly more awards.
- **Camper (coef = 0.5971, p ¡ 0.001):** Participation as a camper increases the expected number of awards.
- **Livebait (coef = 1.8189, p ¡ 0.001):** Students involved in livebait activities have a strong positive effect on awards earned.

Significant Negative Effects

- **Child (coef = -1.1806, p ¡ 0.001):** Having more children is associated with a lower number of awards.

3. Zero-Inflation Model Interpretation

The inflation part explains excess zeros (students with zero awards).

- **Child (positive and significant):** increases probability of being in the zero-award group.
- **Persons and Camper (negative coefficients):** reduce the probability of being a structural zero, meaning they help students move away from zero awards.

4. Key Findings

- The most influential positive predictor of awards is **Livebait participation**.
- **Camper status** and **Persons variable** also significantly increase award counts.
- **Child variable** reduces award counts and increases the chance of zero awards.

Final Conclusion

The number of awards earned by students is significantly influenced by participation-related variables. Students engaged in structured activities such as camping and livebait programs tend to earn more awards, while the presence of competing responsibilities (e.g., child-related factor) reduces academic achievement in terms of awards.

CHAPTER 9

Question No.9

Generalized Linear Model Regression Results

Dep. Variable:	fish_caught	**No. Observations:**	250
Model:	GLM	**Df Residuals:**	245
Model Family:	Binomial	**Df Model:**	4
Link Function:	Logit	**Scale:**	1.0000
Method:	IRLS	**Log-Likelihood:**	-123.83
Date:	Sat, 16 May 2026	**Deviance:**	247.67
Time:	19:25:22	**Pearson chi2:**	231.
No. Iterations:	5	**Pseudo R-squ. (CS):**	0.3141
Covariance Type:	nonrobust		

	coef	**std err**	**z**	**P> |z|**	**[0.025**	**0.975]**
const	-3.1728	0.648	-4.895	0.000	-4.444	-1.902
persons	1.1529	0.197	5.851	0.000	0.767	1.539
child	-2.2203	0.323	-6.871	0.000	-2.854	-1.587
camper	0.9530	0.327	2.910	0.004	0.311	1.595
livebait	0.9768	0.487	2.006	0.045	0.023	1.931

INTERPRETATION

$$\text{const} : -3.1728$$
$$\text{persons} : 1.1529$$
$$\text{child} : -2.2203$$
$$\text{camper} : 0.9530$$
$$\text{livebait} : 0.9768$$

Conclusion:

- Probability of catching fish increases with persons, camper, and livebait.
- Child has a negative effect on fishing success.

Dep. Variable:	fish_caught	**No. Observations:**	250
Model:	GLM	**Df Residuals:**	245
Model Family:	Binomial	**Df Model:**	4
Link Function:	Logit	**Scale:**	1.0000
Method:	IRLS	**Log-Likelihood:**	-123.83
Date:	Sat, 16 May 2026	**Deviance:**	247.67
Time:	19:25:22	**Pearson chi2:**	231.
No. Iterations:	5	**Pseudo R-squ. (CS):**	0.3141
Covariance Type:	nonrobust		

	coef	**std err**	**z**	**P> |z|**	**[0.025**	**0.975]**
const	-3.1728	0.648	-4.895	0.000	-4.444	-1.902
persons	1.1529	0.197	5.851	0.000	0.767	1.539
child	-2.2203	0.323	-6.871	0.000	-2.854	-1.587
camper	0.9530	0.327	2.910	0.004	0.311	1.595
livebait	0.9768	0.487	2.006	0.045	0.023	1.931

Generalized Linear Model Regression Results

Model Interpretation

1. The model indicates that an increase in the number of people (*persons*) in the group significantly increases the likelihood or success rate of catching fish.

2. The presence of a *child* in the group has a significant negative impact on fishing success, noticeably decreasing the probability of catching fish.

3. Both being a *camper* and utilizing *livebait* act as statistically significant positive predictors, substantially boosting the chances of a successful catch.

4. Based on the coefficients, the presence of children carries the strongest overall effect in the model but in a negative direction (coef: -2.2203), presenting a primary obstacle to fishing success.

5. Overall, larger group sizes, choosing to camp, and employing live bait serve as highly effective factors for maximizing the probability of catching fish.

CHAPTER 10

Question No.10

Zero-Truncated Negative Binomial Regression Results

Variable / Parameter	Coefficient (coef)	Incidence Rate Ratio (IRR)
Intercept (const)	1.02185188	2.77833516
Variable 1	-0.00229098	0.99771164
Variable 2	-0.01695145	0.98319141
Variable 3	-0.03037454	0.97008213

Model Interpretation and Estimation Analysis

1. **Analysis of Coefficients (β)**

   The coefficients indicate the directional relationship between the independent variables and the expected log count of the dependent variable:

   - **Intercept (1.0219):** The baseline log count when all independent variables are held at zero.
   - **Variables 1, 2, and 3 (-0.0023, -0.0169, and -0.0304):** All three independent variables display negative coefficients. This systematically indicates that as any of these variable values increase, the expected count of the dependent variable (e.g., days spent in the hospital) decreases.

2. **Analysis of Incidence Rate Ratios (IRR)**

   The Incidence Rate Ratio is calculated by exponentiating the coefficient (e^{β}). It tells us the multiplicative effect of a one-unit change in a predictor variable on the expected rate or count:

   - **Intercept (IRR = 2.7783):** Represents the expected baseline count of the event when all predictors are zero.
   - **Variable 1 (IRR = 0.9977):** A one-unit increase in Variable 1 is associated with a 0.23% decrease in the expected count ($1 - 0.9977 = 0.0023$), holding all other variables constant. This effect is extremely minimal.
   - **Variable 2 (IRR = 0.9832):** A one-unit increase in Variable 2 results in a 1.68% decrease in the expected count ($1 - 0.9832 = 0.0168$).

- **Variable 3 (IRR = 0.9701):** A one-unit increase in Variable 3 results in a 2.99% decrease in the expected count ($1 - 0.9701 = 0.0299$). Variable 3 shows the strongest reductive impact among the predictors.

CHAPTER 11

Question NO.11

Zero-Truncated Poisson Regression Results

Variable / Parameter	Coefficient (coef)	Incidence Rate Ratio (IRR)
Intercept (const)	0.44748491	1.56437270
Variable 1	0.01218667	1.01226126
Variable 2	-0.30134211	0.73982462
Variable 3	0.71647022	2.04719430

Statistical Model Estimation and Interpretation

1. **Analysis of Coefficients (β)**

   The regression coefficients quantify the expected change in the log-transformed count profile of the dependent outcome per unit shift across your set of predictors:

   - **Intercept (0.4475):** This represents the expected baseline log count benchmark context when all concurrent explanatory factors rest at zero.
   - **Variable 1 (0.0122):** Possesses a slight positive coefficient directional signal, meaning an upward change in this indicator subtly correlates with an escalation in the final counted score.
   - **Variable 2 (-0.3013):** Carries a distinct negative coefficient value. This isolates a clear inverse configuration; higher input steps on Variable 2 translate directly to an expected contraction of the primary counter value.
   - **Variable 3 (0.7165):** Holds a prominent positive coefficient magnitude, marking it out as a principal systematic driver that significantly increases the calculated volume metric under study.

2. **Analysis of Incidence Rate Ratios (IRR)**

   The Incidence Rate Ratio metrics are obtained via exponentiation (e^{β}) to reflect a cleaner, direct multiplicative percentage modification vector relative to rate/count structures:

   - **Intercept (IRR = 1.5644):** Evaluates the operational starting rate coefficient index baseline.

- **Variable 1 (IRR** $= 1.0123$**):** A singular step forward along this dimension corresponds with a minor 1.23% incremental increase ($1.0123 - 1 = 0.0123$) across the expected accumulation scale.
- **Variable 2 (IRR** $= 0.7398$**):** A single-unit addition yields a pronounced 26.02% net reduction ($1 - 0.7398 = 0.2602$) over total calculated observation cycles, identifying its core role as an inhibiting variable.
- **Variable 3 (IRR** $= 2.0472$**):** A one-unit structural advancement here pushes a major shift, effectively expanding the anticipated overall frequency occurrence rate by 104.72% (over doubling the original baseline expectations).

CHAPTER 12

Python Code

"'latex

12.1 Code for Question 2

Python Code for One-Way ANOVA Analysis

```

import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# -----------------------------
# Example dataset
# -----------------------------
np.random.seed(42)

# Simulated weight loss data (in pounds)
program_A = np.random.normal(
    loc=5.2,
    scale=1.1,
    size=30
)

program_B = np.random.normal(
    loc=6.8,
    scale=1.3,
    size=30
)

program_C = np.random.normal(
    loc=4.9,
    scale=1.0,
    size=30
)

# Combine into a DataFrame
df = pd.DataFrame({
    'weight_loss': np.concatenate([
        program_A,
        program_B,
        program_C
    ]),
    'program': ['A']*30 + ['B']*30 + ['C']*30
})
```

```python

# ----------------------------
# Step 1: One-way ANOVA
# ----------------------------
model = ols(
    'weight_loss ~ C(program)',
    data=df
).fit()

anova_table = sm.stats.anova_lm(
    model,
    typ=2
)

print("ANOVA Table:\n")
print(anova_table)

# ----------------------------
# Step 2: Assumption Checks
# ----------------------------

# Shapiro-Wilk Test
print("\nShapiro-Wilk Test:")

for prog in ['A', 'B', 'C']:

    stat, p = stats.shapiro(
        df[df['program'] == prog]['weight_loss']
    )

    print(
        f"Program {prog}: "
        f"W={stat:.3f}, "
        f"p={p:.3f}"
    )

# Levenes Test
levene_stat, levene_p = stats.levene(
    program_A,
    program_B,
    program_C
)

print("\nLevenes Test:")
print(
    f"Statistic={levene_stat:.3f}, "
    f"p={levene_p:.3f}"
)

# ----------------------------
# Step 3: Tukey HSD Test
# ----------------------------
tukey = pairwise_tukeyhsd(
    endog=df['weight_loss'],
    groups=df['program'],
    alpha=0.05
)

print("\nTukey HSD Results:\n")
print(tukey)
```

12.2 Code for Question 3

Python Code for Chi-Square Test Analysis

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
import os

# ========================
# Output Folder
# ========================
save_path = (
    r"C:\Users\ayash\OneDrive"
    r"\Desktop\problem3\output"
)

os.makedirs(save_path, exist_ok=True)

# ========================
# Load Dataset
# ========================
df = pd.read_csv(
    r"C:\Users\ayash\OneDrive"
    r"\Desktop\problem3"
    r"\children anemia.csv"
)

# ========================
# Contingency Table
# ========================
table = pd.crosstab(
    df['Age in 5-year groups'],
    df['Anemia level']
)

print("\nContingency Table:\n")
print(table)

# ========================
# Chi-square Test
# ========================
chi2, p, dof, expected = chi2_contingency(table)

print("\nChi-square Results")
print("------------------")

print(f"Chi-square = {chi2:.4f}")
print(f"p-value    = {p:.4f}")
print(f"Degrees of Freedom = {dof}")

```

```python
# ========================
# Save Table to CSV
# ========================
table.to_csv(
    save_path + r"\contingency_table.csv"
)

# Expected Frequencies
expected_df = pd.DataFrame(
    expected,
    index=table.index,
    columns=table.columns
)

expected_df.to_csv(
    save_path + r"\expected_frequencies.csv"
)

# ========================
# Contribution Calculation
# ========================
contribution = (
    (table - expected) ** 2
) / expected

# ========================
# Heatmap Visualization
# ========================
plt.figure(figsize=(10,6))

sns.heatmap(
    contribution,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title(
    "Chi-square Contribution Heatmap",
    fontsize=14
)

plt.xlabel("Anemia Level")
plt.ylabel("Age Group")

# Save Figure
plt.savefig(
    save_path + r"\chi_square_contribution.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

# ========================
# Save Results
# ========================
with open(
    save_path + r"\results.txt",
    "w"
```

```python
) as f:

    f.write("Chi-square Test Results\n")
    f.write("------------------------\n")

    f.write(f"Chi-square: {chi2}\n")
    f.write(f"p-value: {p}\n")
    f.write(f"df: {dof}\n\n")

    if p < 0.05:
        f.write(
            "Conclusion: Significant "
            "association exists.\n"
        )
    else:
        f.write(
            "Conclusion: No significant "
            "association.\n"
        )

print("\nAll outputs saved in:")
print(save_path)
```

12.3 Code for Question 4

Python Code for Fisher Exact Post-hoc Analysis

```python

import pandas as pd
import numpy as np

from scipy.stats import fisher_exact

from statsmodels.stats.multitest import (
    multipletests
)

from itertools import combinations

# ====================================================
# Create Contingency Table
# ====================================================

data = np.array([
    [40, 10],    # Drug A
    [10, 40],    # Drug B
    [25, 25]     # Drug C
])

df = pd.DataFrame(
    data,
    index=[
        'Drug A',
        'Drug B',
        'Drug C'
```

```python
    ],
    columns=[
        'No Disease',
        'Disease'
    ]
)

print("Contingency Table:\n")
print(df)

# ====================================================
# Pairwise Post-hoc Fisher Tests
# ====================================================

pairs = list(
    combinations(df.index, 2)
)

p_values = []

results = []

for pair in pairs:

    sub_table = df.loc[
        list(pair)
    ]

    oddsratio, p = fisher_exact(
        sub_table
    )

    p_values.append(p)

    results.append([
        pair[0],
        pair[1],
        round(oddsratio, 3),
        p
    ])

# ====================================================
# Bonferroni Correction
# ====================================================

adjusted = multipletests(
    p_values,
    method='bonferroni'
)

# ====================================================
# Display Results
# ====================================================

print(
    "\nPost-hoc Fisher Exact "
    "Test Results:\n"
)

for i, res in enumerate(results):
```

```python

    print(
        "Comparison :",
        res[0],
        "vs",
        res[1]
    )

    print(
        "Odds Ratio :",
        res[2]
    )

    print(
        "Raw p-value :",
        round(res[3], 5)
    )

    print(
        "Adjusted p-value :",
        round(adjusted[1][i], 5)
    )

    if adjusted[1][i] < 0.05:

        print(
            "Result : "
            "Significant Difference\n"
        )

    else:

        print(
            "Result : "
            "No Significant Difference\n"
        )
```

“‘latex id=”g5k9ta”

12.4 Code for Question 5

Python Code for Logistic Regression using GLM

```python

# ====================================================
# Step 1: Import Required Libraries
# ====================================================

import pandas as pd

import statsmodels.api as sm

import statsmodels.formula.api as smf


```

```python
# ====================================================
# Step 2: Load Dataset
# ====================================================

url = (
    "https://stats.idre.ucla.edu/"
    "stat/data/binary.csv"
)

df = pd.read_csv(url)


# ====================================================
# Step 3: Preview Dataset
# ====================================================

print(
    "First 5 Rows of Dataset:\n"
)

print(df.head())

print(
    "\nDataset Description:\n"
)

print(df.describe())


# ====================================================
# Step 4: Convert Rank to Categorical
# ====================================================

df['rank'] = df['rank'].astype(
    'category'
)


# ====================================================
# Step 5: Fit Logistic Regression Model
# ====================================================

model = smf.glm(

    formula=
    'admit ~ gre + gpa + rank',

    data=df,

    family=sm.families.Binomial()

).fit()


# ====================================================
# Step 6: Model Summary
# ====================================================

print(
    "\nLogistic Regression "
```

```python
    "Model Summary:\n"
)

print(model.summary())
```

12.5 Code for Question 6

Python Code for Poisson Regression (GLM)

```python

# ==========================================================
# Poisson Regression (GLM)
# Number of Awards Earned by Students
# ==========================================================

# ==========================================================
# Step 1: Import Required Libraries
# ==========================================================

import pandas as pd
import numpy as np

import statsmodels.api as sm

import statsmodels.formula.api as smf


# ==========================================================
# Step 2: Load Dataset
# ==========================================================

url = (
    "https://stats.idre.ucla.edu/"
    "stat/data/poisson_sim.csv"
)

df = pd.read_csv(url)


# ==========================================================
# Step 3: Dataset Information
# ==========================================================

print("\nFirst 5 Rows:\n")

print(df.head())

print("\nDataset Information:\n")

print(df.info())

print("\nSummary Statistics:\n")

print(df.describe())

```

```

# ============================================================
# Step 4: Convert Variable to Category
# ============================================================

df["prog"] = df["prog"].astype(
    "category"
)


# ============================================================
# Step 5: Fit Poisson Regression Model
# ============================================================

model = smf.glm(

    formula=
    "num_awards ~ math + prog",

    data=df,

    family=sm.families.Poisson()

).fit()


# ============================================================
# Step 6: Model Summary
# ============================================================

print(
    "\n=============== "
    "POISSON REGRESSION SUMMARY "
    "===============\n"
)

print(model.summary())


# ============================================================
# Step 7: Incidence Rate Ratios (IRR)
# ============================================================

irr = np.exp(model.params)

print(
    "\n=============== "
    "INCIDENCE RATE RATIOS (IRR) "
    "===============\n"
)

print(irr)


# ============================================================
# Step 8: Predicted Counts
# ============================================================

df["Predicted_Count"] = model.predict(df)

```

```
print(
    "\n=============== "
    "PREDICTED NUMBER OF AWARDS "
    "===============\n"
)

print(
    df[
        [
            "num_awards",
            "Predicted_Count"
        ]
    ].head(10)
)


# ========================================================
# Step 9: Interpretation
# ========================================================

print(
    "\n=============== "
    "INTERPRETATION "
    "===============\n"
)

for variable, pvalue in model.pvalues.items():

    if pvalue < 0.05:

        significance = (
            "Statistically Significant"
        )

    else:

        significance = (
            "Not Significant"
        )

    print(
        f"{variable} "
        f"--> p-value = "
        f"{pvalue:.4f} "
        f"--> {significance}"
    )


print("\nConclusion:")

print("""

1. Math score has a positive
   effect on the number of
   awards earned.

2. Students in different
   academic programs earn
   different numbers of awards.

```

```
3. Significant predictors
   (p < 0.05) strongly
   influence award counts.

4. IRR > 1 indicates increased
   expected award counts.

5. IRR < 1 indicates decreased
   expected award counts.

""")
```

12.6 Code for Question 7

Python Code for Negative Binomial Regression

```python

# =========================================================
# Negative Binomial Regression (GLM)
# High School Student Absenteeism Dataset
# =========================================================

# =========================================================
# Step 1: Import Libraries
# =========================================================

import pandas as pd
import numpy as np

import statsmodels.api as sm

import statsmodels.formula.api as smf


# =========================================================
# Step 2: Load Dataset
# =========================================================

url = (
    "https://stats.idre.ucla.edu/"
    "stat/stata/dae/nb_data.dta"
)

df = pd.read_stata(url)


# =========================================================
# Step 3: Preview Dataset
# =========================================================

print("\nFirst 5 Rows:\n")

print(df.head())

print("\nDataset Information:\n")
```

```python

print(df.info())

print("\nSummary Statistics:\n")

print(df.describe())


# ==========================================================
# Step 4: Convert Variable to Category
# ==========================================================

df["prog"] = df["prog"].astype(
    "category"
)


# ==========================================================
# Step 5: Fit Negative Binomial Model
# ==========================================================

model = smf.glm(

    formula=
    "daysabs ~ math + prog",

    data=df,

    family=sm.families.NegativeBinomial()

).fit()


# ==========================================================
# Step 6: Model Summary
# ==========================================================

print(
    "\n=============== "
    "NEGATIVE BINOMIAL "
    "MODEL SUMMARY "
    "===============\n"
)

print(model.summary())


# ==========================================================
# Step 7: Incidence Rate Ratios (IRR)
# ==========================================================

irr = np.exp(model.params)

print(
    "\n=============== "
    "INCIDENCE RATE RATIOS (IRR) "
    "===============\n"
)

print(irr)
```

```python


# =========================================================
# Step 8: Predicted Absence Counts
# =========================================================

df["Predicted_Absence"] = model.predict(df)

print(
    "\n=============== "
    "PREDICTED DAYS OF ABSENCE "
    "===============\n"
)

print(
    df[
        [
            "daysabs",
            "Predicted_Absence"
        ]
    ].head(10)
)


# =========================================================
# Step 9: Interpretation
# =========================================================

print(
    "\n=============== "
    "INTERPRETATION "
    "===============\n"
)

for variable, pvalue in model.pvalues.items():

    if pvalue < 0.05:

        significance = (
            "Statistically Significant"
        )

    else:

        significance = (
            "Not Significant"
        )

    print(
        f"{variable} "
        f"--> p-value = "
        f"{pvalue:.4f} "
        f"--> {significance}"
    )


print("\nConclusion:")

print("""

```

```
1. Math score significantly
   affects student absenteeism.

2. Different academic programs
   are associated with different
   absence rates.

3. Significant predictors
   (p < 0.05) strongly influence
   the number of absent days.

4. IRR > 1 means increased
   absence rate.

5. IRR < 1 means decreased
   absence rate.

6. Negative Binomial Regression
   is appropriate because count
   data often show over-dispersion
   (variance > mean).

""")
```

12.7 Code for Question 8

Python Code for Zero-Inflated Poisson (ZIP) Regression

```python

# ========================================================
# Zero-Inflated Poisson (ZIP) Regression
# Fish Catch Dataset
# ========================================================

# ========================================================
# Step 1: Import Libraries
# ========================================================

import pandas as pd

import statsmodels.api as sm

from statsmodels.discrete.count_model import (
    ZeroInflatedPoisson
)


# ========================================================
# Step 2: Load Dataset
# ========================================================

url = (
    "https://stats.idre.ucla.edu/"
    "stat/data/fish.csv"
)
```

```python

df = pd.read_csv(url)


# ==========================================================
# Step 3: Check Dataset Columns
# ==========================================================

print("\nDataset Columns:\n")

print(df.columns)


# ==========================================================
# Step 4: Define Response Variable
# ==========================================================

y = df["count"]


# ==========================================================
# Step 5: Define Predictor Variables
# ==========================================================

X = df[
    [
        "persons",
        "child",
        "camper",
        "livebait"
    ]
]

X = sm.add_constant(X)


# ==========================================================
# Step 6: Inflation Model
# ==========================================================

X_infl = X


# ==========================================================
# Step 7: Fit ZIP Regression Model
# ==========================================================

zip_model = ZeroInflatedPoisson(

    endog=y,

    exog=X,

    exog_infl=X_infl,

    inflation="logit"

)

```

```python
result = zip_model.fit(

    method="bfgs",

    maxiter=200

)


# =========================================================
# Step 8: Model Summary
# =========================================================

print(
    "\n=============== "
    "ZERO-INFLATED POISSON "
    "MODEL SUMMARY "
    "===============\n"
)

print(result.summary())
```

12.8 Code for Question 9

Python Code for Zero-Inflated Poisson Model

```python

# =========================================================
# Zero-Inflated Poisson (ZIP) Regression
# Fish Catch Dataset
# =========================================================

# =========================================================
# Step 1: Import Libraries
# =========================================================

import pandas as pd

import statsmodels.api as sm

from statsmodels.discrete.count_model import (
    ZeroInflatedPoisson
)


# =========================================================
# Step 2: Load Dataset
# =========================================================

url = (
    "https://stats.idre.ucla.edu/"
    "stat/data/fish.csv"
)

df = pd.read_csv(url)
```

```


# ============================================================
# Step 3: Define Response Variable
# ============================================================

y = df["count"]


# ============================================================
# Step 4: Define Predictor Variables
# ============================================================

X = df[
    [
        "persons",
        "child",
        "camper",
        "livebait"
    ]
]

X = sm.add_constant(X)


# ============================================================
# Step 5: Fit ZIP Regression Model
# ============================================================

model = ZeroInflatedPoisson(

    endog=y,

    exog=X,

    exog_infl=X,

    inflation='logit'

)


result = model.fit(

    method="bfgs",

    maxiter=200

)


# ============================================================
# Step 6: Display Model Summary
# ============================================================

print(
    "\n=============== "
    "ZERO-INFLATED POISSON "
    "MODEL SUMMARY "
    "===============\n"
```

```
)

print(result.summary())
```

12.9 Code for Question 10

Python Code for Zero-Truncated Poisson (ZTP) Regression

```

# ============================================================
# Zero-Truncated Poisson (ZTP) Regression
# Synthetic Healthcare Dataset
# ============================================================

# ============================================================
# Step 1: Import Required Libraries
# ============================================================

import numpy as np

import pandas as pd

import statsmodels.api as sm

from scipy.optimize import minimize

from scipy.special import gammaln


# ============================================================
# Step 2: Create Synthetic Dataset
# ============================================================

np.random.seed(123)

n = 300

age = np.random.randint(
    20,
    80,
    n
)

insurance = np.random.choice(
    [0, 1, 2],
    n
)

death = np.random.choice(
    [0, 1],
    n
)


# ============================================================
```

```
# Step 3: Design Matrix
# ==========================================================

X = np.column_stack([
    age,
    insurance,
    death
])

X = sm.add_constant(X)


# ==========================================================
# Step 4: True Coefficients
# ==========================================================

beta_true = np.array([
    0.2,
    0.01,
    -0.15,
    0.5
])


# ==========================================================
# Step 5: Generate Poisson Mean
# ==========================================================

mu = np.exp(
    X @ beta_true
)


# ==========================================================
# Step 6: Generate Poisson Counts
# ==========================================================

y = np.random.poisson(mu)


# ==========================================================
# Step 7: Remove Zero Counts
# ==========================================================

y = y[y > 0]

X = X[:len(y)]


# ==========================================================
# Step 8: Define ZTP Log-Likelihood
# ==========================================================

def ztp_loglike(
    params,
    y,
    X
):

    eta = X @ params
```

```
    mu = np.exp(eta)

    loglik = (
        y * np.log(mu)
        - mu
        - gammaln(y + 1)
    )

    loglik -= np.log(
        1 - np.exp(-mu)
    )

    return -np.sum(loglik)


# ========================================================
# Step 9: Optimization
# ========================================================

init = np.zeros(
    X.shape[1]
)

res = minimize(

    ztp_loglike,

    init,

    args=(y, X),

    method="BFGS"

)


# ========================================================
# Step 10: Display Results
# ========================================================

print("\nCoefficients:\n")

print(res.x)


# ========================================================
# Step 11: Incidence Rate Ratios (IRR)
# ========================================================

print(
    "\nIncidence Rate Ratios (IRR):\n"
)

print(
    np.exp(res.x)
)
```

12.10 Code for Question 11

Python Code for Zero-Truncated Negative Binomial Regression

```

# ==========================================================
# Zero-Truncated Negative Binomial (ZTNB) Regression
# Simulated Hospital Dataset
# ==========================================================

# ==========================================================
# Step 1: Import Required Libraries
# ==========================================================

import numpy as np

import pandas as pd

import statsmodels.api as sm

from scipy.optimize import minimize

from scipy.special import gammaln


# ==========================================================
# Step 2: Simulate Hospital Dataset
# ==========================================================

np.random.seed(42)

n = 400


age = np.random.randint(
    18,
    90,
    n
)

insurance = np.random.choice(
    [0, 1, 2],
    n
)

death = np.random.choice(
    [0, 1],
    n
)


# ==========================================================
# Step 3: Create Design Matrix
# ==========================================================

X = np.column_stack([
    age,
    insurance,
    death
```

```
])

X = sm.add_constant(X)


# ==========================================================
# Step 4: Define True Parameters
# ==========================================================

beta = np.array([
    0.3,
    0.015,
    -0.25,
    0.8
])


# ==========================================================
# Step 5: Generate Mean Counts
# ==========================================================

mu = np.exp(
    X @ beta
)


# ==========================================================
# Step 6: Generate Negative Binomial Counts
# ==========================================================

y = np.random.negative_binomial(

    1,

    1 / (1 + mu)

)


# ==========================================================
# Step 7: Zero Truncation
# ==========================================================

mask = y > 0

y = y[mask]

X = X[mask]


# ==========================================================
# Step 8: Define ZTNB Log-Likelihood
# ==========================================================

def ztnb_loglike(
    params,
    y,
    X,
    alpha=1.0
):
```

```
    eta = X @ params

    mu = np.exp(eta)

    loglik = (

        gammaln(y + 1/alpha)

        - gammaln(y + 1)

        - gammaln(1/alpha)

        + y * np.log(mu)

        - (y + 1/alpha)
        * np.log(1 + alpha * mu)

    )

    p0 = (
        1 + alpha * mu
    ) ** (-1 / alpha)

    loglik -= np.log(
        1 - p0
    )

    return -np.sum(loglik)


# ========================================================
# Step 9: Fit ZTNB Model
# ========================================================

init = np.zeros(
    X.shape[1]
)

res = minimize(

    ztnb_loglike,

    init,

    args=(y, X),

    method="BFGS"

)


# ========================================================
# Step 10: Display Coefficients
# ========================================================

print("\nCoefficients:\n")

print(res.x)

```

```python

# ========================================================
# Step 11: Incidence Rate Ratios (IRR)
# ========================================================

print(
    "\nIncidence Rate Ratios (IRR):\n"
)

print(
    np.exp(res.x)
)
```